PROYECTO DE CLASIFICACIÓN DE SEÑALES DE AUDIO CON RNN Y CNN

Carol Valentina Castellanos Jaimes
Estudiante de Ingenieria Electronica
Universidad del Magdalena
Santa Marta, Magdalena
cvcastellanos@unimagdalena.edu.co

Franllely Daniela Marquina Rodríguez
Estudiante de Ingenieria Electronica
Universidad del Magdalena
Santa Marta, Magdalena
fmarquina@unimagdalena.edu.co

***Resumen*—** En este proyecto se implementó un sistema de clasificación de género por voz utilizando técnicas de procesamiento digital de señales y aprendizaje profundo. Se empleó un dataset público de audios de voz masculina y femenina, realizando análisis temporal y frecuencial mediante forma de onda, FFT y mel-espectrogramas. Posteriormente, se desarrollaron dos enfoques de clasificación: una red neuronal convolucional (CNN) basada en mel-espectrogramas y una red neuronal recurrente (RNN-LSTM) utilizando coeficientes MFCC como representación secuencial del audio. Los resultados permitieron comparar ambos modelos para identificar el enfoque más adecuado en la clasificación automática de señales de voz.

***Palabras Claves*—** CNN, RNN, LSTM, FFT, MFCC, Mel-espectrograma, Clasificación de voz, Procesamiento Digital de Señales.

Abstract— This project implemented a voice gender classification system using digital signal processing and deep learning techniques. A public dataset of male and female voice recordings was used, performing time and frequency analysis using waveform analysis, FFTs, and mel-spectrograms. Subsequently, two classification approaches were developed: a convolutional neural network (CNN) based on mel-spectrograms and a recurrent neural network (RNN-LSTM) using MFCC coefficients as a sequential representation of the audio. The results allowed for a comparison of both models to identify the most suitable approach for the automatic classification of voice signals.

Keywords— CNN, RNN, LSTM, FFT, MFCC, Mel-spectrogram, Speech classification, Digital Signal Processing.

I. MARCO TEORICO

El procesamiento digital de señales (PDS) permite analizar, transformar e interpretar señales discretas obtenidas de fenómenos físicos, como la voz humana. En este proyecto se implementó un sistema de clasificación de género por voz utilizando redes neuronales profundas basadas en dos enfoques: Redes Neuronales Convolucionales (CNN) y Redes Neuronales Recurrentes (RNN). El objetivo principal fue comparar el desempeño de ambos modelos utilizando representaciones temporales y espectrales de señales de audio.

A. Señales de Audio Digitales

Una señal de audio digital es una representación discreta de una señal analógica obtenida mediante muestreo. El proceso de muestreo consiste en tomar valores de amplitud de la señal continua a intervalos regulares definidos por la frecuencia de muestreo fs. En este proyecto se trabajó con audios remuestreados a:

$$f_s = 22050\,Hz \quad (1)$$

La señal discreta puede expresarse como:

$$x[n] = x(nT_s) \quad (2)$$

donde T_s es el período de muestreo y n representa el índice temporal discreto.

Las propiedades principales analizadas fueron duración, amplitud máxima, frecuencia de muestreo y número de canales, parámetros fundamentales para garantizar uniformidad en el procesamiento de los audios.

B. Transformada de Fourier y Análisis Espectral

El análisis espectral permite estudiar las frecuencias presentes en una señal de voz. Para ello se empleó la Transformada Discreta de Fourier (DFT), calculada mediante el algoritmo FFT (Fast Fourier Transform). La DFT se define como:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N} \quad (3)$$

Esta transformación convierte la señal del dominio del tiempo al dominio de la frecuencia, permitiendo identificar componentes espectrales dominantes. En voces humanas, las frecuencias fundamentales suelen encontrarse aproximadamente entre 85–180 Hz para voces masculinas y 165–255 Hz para voces femeninas, lo que facilita su clasificación automática.

El espectro de magnitud utilizado en el proyecto se obtiene mediante:

$$|\, X[k] \,| = \sqrt{Re\{X[k]\}^2 + Im\{X[k]\}^2} \quad (4)$$

C. Espectrogramas y Escala Mel

Un espectrograma es una representación tiempo-frecuencia que muestra cómo evolucionan las frecuencias de una señal a lo largo del tiempo. Para su generación se utiliza la Transformada de Fourier de Tiempo Corto (STFT):

$$X(m,w) = \sum_{n=-\infty}^{\infty} x[n]w[n-m]\, e^{-jwn} \quad (5)$$

donde $w[n]$ corresponde a una ventana temporal aplicada a segmentos de la señal.

En este proyecto se emplearon mel-espectrogramas, los cuales transforman las frecuencias hacia la escala Mel, diseñada para aproximarse a la percepción auditiva humana. La conversión aproximada entre frecuencia y escala Mel es:

$$Mel(f) = 2595 log_{10}\left(1 + \frac{f}{700}\right) \quad (6)$$

Los mel-espectrogramas fueron utilizados como entrada del modelo CNN debido a que permiten representar patrones visuales relacionados con energía, tono y distribución frecuencial de la voz.

D. Coeficientes MFCC

Los MFCC (Mel Frequency Cepstral Coefficients) son características ampliamente utilizadas en reconocimiento de voz debido a que condensan la información más relevante del espectro de audio. Estos coeficientes se obtienen aplicando la transformada logarítmica y posteriormente la Transformada Discreta del Coseno (DCT):

$$C_n = \sum_{k=1}^{K} log(S_k) cos[n(k - \tfrac{1}{2})\tfrac{\pi}{K}] \quad (7)$$

donde S_k representa la energía de cada banda Mel. En el proyecto se utilizaron 13 coeficientes MFCC como representación secuencial para el modelo RNN/LSTM.

E. Redes Neuronales Convolucionales (CNN)

Las CNN son modelos de aprendizaje profundo especializados en el análisis de imágenes y patrones espaciales. En este trabajo, los mel-espectrogramas fueron tratados como imágenes bidimensionales de tamaño 64×70.

La operación fundamental de una CNN es la convolución:

$$((f * g)(t) = \sum_{\tau=-\infty}^{\infty} f(\tau) g(t-\tau)) \quad (8)$$

Las capas convolucionales permiten extraer características relevantes como patrones de frecuencia y energía, mientras que las capas de max pooling reducen dimensionalidad y complejidad computacional. Finalmente, la capa sigmoide realiza la clasificación binaria entre voz masculina y femenina mediante:

$$\sigma(x) = \frac{1}{1+e^{-x}} \quad (9)$$

F. Redes Neuronales Recurrentes (RNN y LSTM)

Las RNN son modelos diseñados para procesar secuencias temporales, conservando información previa mediante estados internos. Su funcionamiento básico puede expresarse como:

$$h_t = f(Wx_t + Uh_{t-1} + b) \quad (10)$$

donde h_t representa el estado oculto actual y h_{t-1} el estado anterior.

En este proyecto se utilizó una arquitectura LSTM (Long Short-Term Memory), una variante de RNN capaz de manejar dependencias temporales largas y evitar el problema del desvanecimiento del gradiente. Este modelo analizó la evolución temporal de los coeficientes MFCC para identificar patrones característicos de cada género.

G. Métricas de Evaluación

El desempeño de los modelos fue evaluado utilizando accuracy y matriz de confusión. La exactitud o precisión global se calcula mediante:

$$Accuracy = \frac{TP+TN}{FP+FNTP+TN} \quad (11)$$

donde:

TP: verdaderos positivos.

TN: verdaderos negativos.

FP: falsos positivos.

FN: falsos negativos.

La matriz de confusión permitió identificar errores de clasificación y comparar objetivamente el desempeño entre CNN y RNN.

II. DESARROLLO

A. Dataset e inserción de datos

- Código implementado

Se configuró el acceso a la API de Kaggle importando el archivo kaggle.json desde Google Colab, creando la carpeta .kaggle y asignando los permisos necesarios para establecer una conexión segura.

Figura 1. Código de carga y acceso al dataset

Se instaló la librería kaggle, se descargó el dataset Gender Recognition by Voice y se descomprimió el archivo .zip para acceder a los audios .wav.

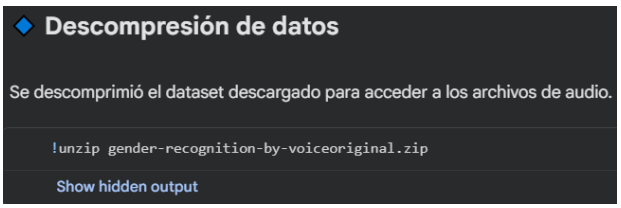

Figura 2. Código de descomprensión de datos

Mediante la librería os se verificó la estructura del dataset, confirmando la organización en dos carpetas (male y female), y se contabilizaron las muestras disponibles por clase para conocer la distribución inicial de los datos.

Figura 3.Código de exploración del dataset

- Descripción del dataset

Para el desarrollo del proyecto se utilizó el dataset Gender Recognition by Voice de Kaggle, compuesto por audios .wav etiquetados en dos clases: voces masculinas y femeninas. Se seleccionaron 200 muestras balanceadas (100 por clase), procesadas a 22 050 Hz con duración aproximada de 4 segundos, y divididas en 70% entrenamiento, 20% validación y 10% prueba.

B. Preparación de datos

- Código implementado

Se importaron las librerías random y train_test_split para la selección y división de datos. Se generaron las rutas de los archivos .wav de ambas clases y se fijó una semilla aleatoria (random.seed(42)) para garantizar la reproducibilidad en la selección de muestras.

Se seleccionaron aleatoriamente 100 audios por clase, conformando un conjunto balanceado de 200 muestras. Las etiquetas se codificaron asignando 0 a voces femeninas y 1 a voces masculinas.

Figura 4. Código de preparación de datos para el modelo

Los datos se dividieron en tres subconjuntos mediante train_test_split: 10 % para prueba final, y el resto

distribuido entre entrenamiento y validación, manteniendo la proporción de clases con el parámetro stratify. Para el procesamiento y extracción de características de audio se emplearon las librerías librosa y numpy.

```
División de los datos (entrenamiento/ validacion / prueba)

X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.10, stratify=y, random_state=42
)

X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=40, stratify=y_temp, random_state=42
)

print("Entrenamiento:", len(X_train))
print("Validación:", len(X_val))
print("Prueba final:", len(X_test))

Entrenamiento: 140
Validación: 40
Prueba final: 20

Procesamiento de audio

import librosa
import numpy as np
```

Figura 5. Codigo de division de datos (entrenamiento/validacion/prueba)

- Descripción de los datos

Se generaron las rutas de los archivos .wav de cada clase y se cargaron con Librosa. Se aplicó padding para uniformizar la longitud de las señales y se asignaron etiquetas binarias (0: femenino, 1: masculino) como variables de entrada para los modelos.

C. Espectrograma

- Código implementado

Cada audio fue convertido a mel-espectrograma mediante librosa, usando 22 050 Hz de muestreo y 64 bandas mel. La energía se transformó a escala logarítmica (dB) y se aplicó padding o recorte para normalizar todas las entradas al tamaño fijo de 64×70, garantizando uniformidad en la entrada del modelo CNN.

```
Conversión de audio a espectrograma

def audio_to_melspectrogram(path, max_pad_len=70):
    audio, sr = librosa.load(path, sr=22050)
    mel = librosa.feature.melspectrogram(y=audio, sr=sr, n_mels=64)
    mel_db = librosa.power_to_db(mel, ref=np.max)

    if mel_db.shape[1] < max_pad_len:
        pad_width = max_pad_len - mel_db.shape[1]
        mel_db = np.pad(mel_db, pad_width=((0,0), (0,pad_width)), mode='constant',constant_values=-80)
    else:
        mel_db = mel_db[:, :max_pad_len]

    return mel_db

X_train_cnn = np.array([audio_to_melspectrogram(p) for p in X_train])
X_val_cnn = np.array([audio_to_melspectrogram(p) for p in X_val])
X_test_cnn = np.array([audio_to_melspectrogram(p) for p in X_test])

X_train_cnn = X_train_cnn[..., np.newaxis]
X_val_cnn = X_val_cnn[..., np.newaxis]
X_test_cnn = X_test_cnn[..., np.newaxis]

y_train = np.array(y_train)
y_val = np.array(y_val)
y_test = np.array(y_test)

print(X_train_cnn.shape)

import matplotlib.pyplot as plt
```

Figura 6. Código del mel-espectrograma

Los mel-espectrogramas se almacenaron en arreglos NumPy y se añadió un canal adicional con np.newaxis para compatibilidad con la CNN. Se visualizó un ejemplo con matplotlib, donde los tonos amarillos representan mayor energía y los oscuros menor intensidad frecuencial.

```
# Elegir un audio de ejemplo
audio_ejemplo = X_train[100]


visual_max_pad_len = 70 # Puedes ajustar este valor, por ejemplo, 60, 90, o mantener 128

# Generar espectrograma con la longitud de padding ajustada para el ejemplo
spec = audio_to_melspectrogram(audio_ejemplo, max_pad_len=visual_max_pad_len)

# Mostrar imagen
plt.figure(figsize=(6,4))
import librosa.display

plt.figure(figsize=(6,4))

librosa.display.specshow(
    spec,
    sr=22050,
    x_axis='time',
    y_axis='hz',
    cmap='magma'
)

plt.title("Espectrograma en Hz")
plt.xlabel("Tiempo")
plt.ylabel("Frecuencia (Hz)")
plt.colorbar(format='%+2.0f dB')
plt.show()
```

Figura 7. Código generador del espectrograma

- Descripción del espectrograma

El mel-espectrograma transforma la señal de audio a escala Mel y decibelios (dB) para aproximar la percepción auditiva humana. Las matrices resultantes fueron ajustadas a dimensiones fijas de 64×70×1 mediante padding y recorte, donde las filas representan frecuencias, las columnas el tiempo y el canal corresponde a escala de grises. En la visualización, los tonos amarillos indican alta energía, los verdes niveles intermedios y los azules o morados baja intensidad.

- Grafica del espectrograma

El espectrograma presentado muestra la distribución de energía de una señal de voz en función del tiempo y la frecuencia. Se observa una mayor concentración energética en las bajas frecuencias, principalmente por debajo de los 3000 Hz, donde aparecen bandas horizontales brillantes asociadas a los armónicos característicos de la voz humana. A medida que aumenta la frecuencia, la intensidad disminuye progresivamente, evidenciada por regiones más oscuras.

También se identifican variaciones temporales en la energía de la señal, reflejadas en cambios de color y en interrupciones verticales que corresponden a pausas o transiciones durante el habla. La barra lateral en decibelios indica el nivel de intensidad espectral, donde valores cercanos a −10 dB representan componentes dominantes y valores próximos a −80 dB corresponden a regiones de muy baja energía o silencio.

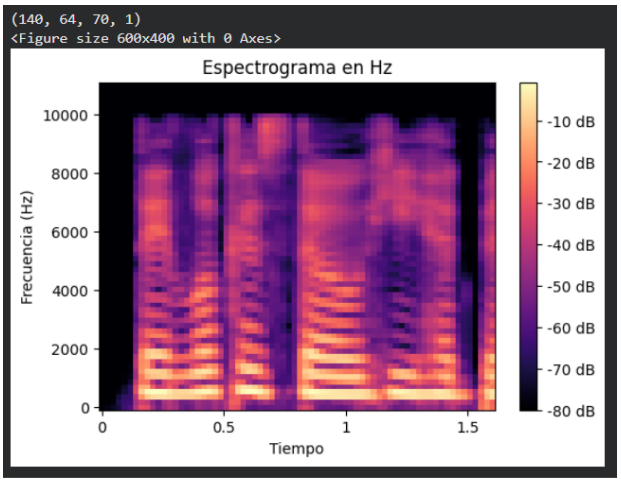

Figura 8. Grafica del espectrograma

D. *Espectro de frecuencia*

- Código implementado

Se emplearon NumPy, Matplotlib y Librosa para el procesamiento y visualización de la señal. Un archivo del conjunto de entrenamiento fue cargado con librosa.load a una frecuencia de muestreo de 22 050 Hz.

Espectro de frecuencia

```python
import numpy as np
import matplotlib.pyplot as plt
import librosa
import librosa.display # Ensure librosa.display is available for plotting audio waveforms

# Choose an example audio file path from the training set
# Assuming X_train is available from previous cells
# You can change the index to try different audio files, e.g., X_train[0], X_train[1], etc.
# Using the same audio_ejemplo that was used for the spectrogram

# Define audio_ejemplo to resolve NameError, using an example from X_train
audio_ejemplo = X_train[100] # Using the same index as where it was previously defined

audio_file_path = audio_ejemplo

print(f"Processing audio file: {audio_file_path}")

# Load the audio file
audio, sr = librosa.load(audio_file_path, sr=22050) # sr=22050 is commonly used for audio processing

# --- 1. Plot Original Audio Waveform ---
plt.figure(figsize=(12, 4))
librosa.display.waveshow(y=audio, sr=sr)
plt.title('Audio Waveform (Time Domain)')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.grid(True)
plt.tight_layout()
plt.show()
```

Figura 9. Código de la gráfica tiempo vs amplitud

Se visualizó la forma de onda con librosa.display.waveshow para observar variaciones de amplitud e intensidad. Posteriormente, se aplicó la FFT mediante np.fft.fft para transformar la señal al dominio frecuencial, considerando únicamente la mitad positiva del espectro por simetría, lo que permitió identificar las componentes frecuenciales dominantes de la voz analizada.

```python
# --- 2. Compute DFT (using FFT algorithm) and plot Magnitude Spectrum ---

# Number of samples in the audio signal
N = len(audio)

# Compute the Fast Fourier Transform (FFT) which is an efficient algorithm for DFT
yf = np.fft.fft(audio)

# Compute the corresponding frequencies
# The frequencies are symmetric around 0, from -sr/2 to sr/2
xf = np.fft.fftfreq(N, 1 / sr)

# We are interested in the magnitude of the positive frequency components
# The FFT output is symmetric, so we only take the first half (up to N//2)
# Multiply by 2.0/N to normalize the magnitude (excluding DC and Nyquist if N is even)
yf_magnitude = 2.0/N * np.abs(yf[0:N//2])
xf_positive = xf[0:N//2]

# Plot the Magnitude Spectrum
plt.figure(figsize=(12, 4))
plt.plot(xf_positive, yf_magnitude)
plt.title('Magnitude Spectrum (DFT/FFT - Frequency Domain)')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.grid(True)
plt.xlim(0, sr/2) # Limit x-axis to Nyquist frequency (max meaningful frequency)
plt.tight_layout()
plt.show()
```

Figura 10. Código de la gráfica frecuencia vs magnitud

- Descripción del análisis espectral mediante FFT/DFT

La FFT permitió transformar las señales al dominio frecuencial y obtener el espectro de magnitud, revelando diferencias entre voces masculinas y femeninas a través de sus componentes frecuenciales dominantes. Este análisis complementó la información temporal con la frecuencial, facilitando la identificación de patrones acústicos relevantes para la clasificación automática.

- Grafica tiempo vs amplitud

La gráfica de la forma de onda en el dominio del tiempo muestra la variación de la amplitud de la señal de audio a lo largo de aproximadamente 4 segundos. Se observan segmentos con mayor intensidad y actividad en diferentes intervalos temporales, evidenciando cambios en la energía de la voz durante la pronunciación. Los picos positivos y negativos representan las oscilaciones propias de la señal sonora, mientras que las zonas cercanas a cero indican pausas o menor presencia de sonido. La amplitud alcanza valores relativamente altos en ciertos momentos, lo que refleja partes de mayor énfasis o potencia vocal. En general, la señal presenta un comportamiento dinámico y no estacionario, característico de las señales de voz humana en el dominio temporal.

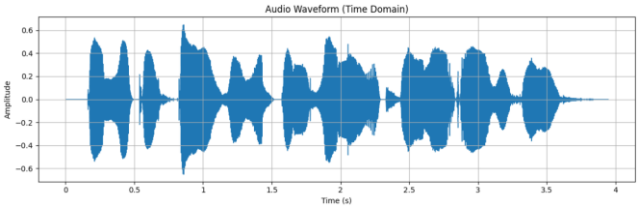

Figura 11. Grafica tiempo vs amplitud

- Grafica frecuencia vs amplitud

La gráfica del espectro de magnitud evidencia una mayor concentración de energía en las bajas frecuencias,

donde se observan los picos principales asociados a la frecuencia fundamental y armónicos de la voz. A medida que la frecuencia aumenta, la amplitud disminuye progresivamente, mostrando una menor presencia energética en frecuencias altas. La mayor parte de la información relevante de la señal se concentra por debajo de los 2000 Hz, rango importante para el análisis y diferenciación de características vocales.

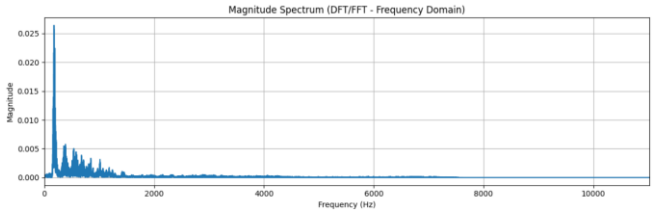

Figura 12. Grafica frecuencia vs magnitud

E. Propiedades de la señal de audio

- Código implementado

Se calculó la duración del audio mediante len(audio)/sr, se identificó que la señal es monocanal y se obtuvo la amplitud máxima con np.max(np.abs(audio)), parámetro útil para evaluar el nivel de energía y determinar posibles procesos de normalización previos al análisis.

Análisis básico de propiedades de la señal de audio

Double-click (or enter) to edit

```
print("Duración:", len(audio)/sr)
print("Canales:", 1)
print("Amplitud máxima:", np.max(np.abs(audio)))
```

```
Duración: 3.9450340136054423
Canales: 1
Amplitud máxima: 0.6497705
```

La señal analizada tiene una duración de 3.95 segundos, lo que representa el tiempo total de la grabación, y un solo canal (mono), indicando que toda la información de audio está contenida en una única secuencia temporal. La amplitud máxima de 0.6497705 refleja la mayor intensidad alcanzada por la señal, siendo un parámetro clave para evaluar su nivel de energía y posible normalización antes del procesamiento.

Figura 13. Código de las propiedades del análisis

- Descripción del análisis

El análisis temporal permite visualizar la forma de onda de la señal, identificando características como duración, amplitud máxima y número de canales. La forma de onda refleja las variaciones de intensidad de la voz, evidenciando segmentos de mayor energía, pausas y cambios dinámicos propios del habla.

F. Construcción del modelo CNN

- Código implementado

Se construyó una CNN con TensorFlow Keras mediante una arquitectura secuencial compuesta por dos bloques Conv2D+MaxPooling2D (32 y 64 filtros de 3×3 con activación ReLU), seguidos de una capa Flatten, una capa densa de 64 neuronas y una capa de salida con activación sigmoide para clasificación binaria. El modelo se compiló con el optimizador Adam, función de pérdida binary_crossentropy y métrica accuracy.

Construcción del modelo CNN

```
from tensorflow.keras import models, layers

cnn_model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(64,70,1)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

cnn_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

cnn_model.summary()
```

Figura 14. Código de la construcción del modelo CNN

- Descripción de la construcción

La implementación del modelo CNN se basó en una arquitectura orientada al reconocimiento de patrones en espectrogramas de audio. Las capas convolucionales permiten identificar características acústicas relevantes, mientras que las capas MaxPooling reducen la dimensionalidad de la información para mejorar la eficiencia del modelo. Se utilizó la función ReLU para acelerar el aprendizaje y una capa sigmoide para generar la clasificación binaria final. El entrenamiento se configuró con el optimizador Adam, función de pérdida binary_crossentropy, un tamaño de lote de 16 muestras y 10 épocas de entrenamiento.

- Tabla de tipos de parámetros en CNN

La arquitectura CNN incrementa progresivamente los filtros de 32 a 64 para extraer características acústicas más complejas, mientras las capas MaxPooling2D reducen la dimensionalidad espacial. La capa Flatten genera un vector de 14 336 valores que alimenta las capas densas, concentrando la mayor cantidad de parámetros entrenables en la primera capa densa. El modelo totaliza 936 449 parámetros entrenables.

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 68, 32)	320
max_pooling2d (MaxPooling2D)	(None, 31, 34, 32)	0
conv2d_1 (Conv2D)	(None, 29, 32, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 14, 16, 64)	0
flatten (Flatten)	(None, 14336)	0
dense_2 (Dense)	(None, 64)	917,568
dense_3 (Dense)	(None, 1)	65

Total params: 936,449 (3.57 MB)
Trainable params: 936,449 (3.57 MB)
Non-trainable params: 0 (0.00 B)

Figura 15. Tabla de los parámetros del CNN

G. *Entrenamiento del modelo CNN*

- Código implementado

El modelo CNN se entrenó con el método fit() durante 10 épocas y un batch size de 16, usando los espectrogramas como entrada. En cada época se evaluó el desempeño sobre el conjunto de validación sin modificar los pesos, permitiendo monitorear la capacidad de generalización del modelo.

Entrenamiento del modelo CNN

```
history_cnn = cnn_model.fit(
    X_train_cnn, y_train,
    validation_data=(X_val_cnn, y_val),
    epochs=10,
    batch_size=16
)
```

Figura 16. Código de entrenamiento del modelo CNN

Mediante matplotlib se graficaron las curvas de precisión de entrenamiento y validación extraídas del objeto history_cnn, permitiendo visualizar la evolución del aprendizaje del modelo CNN y evaluar su capacidad de generalización a lo largo de las 10 épocas.

```
import matplotlib.pyplot as plt

# Obtener los datos de historial para el modelo CNN
acc_cnn = history_cnn.history['accuracy']
val_acc_cnn = history_cnn.history['val_accuracy']
epochs_range = range(len(acc_cnn))

plt.figure(figsize=(8, 6))

# Gráfico para el modelo CNN
plt.plot(epochs_range, acc_cnn, label='Precisión de Entrenamiento CNN')
plt.plot(epochs_range, val_acc_cnn, label='Precisión de Validación CNN')
plt.legend(loc='lower right')
plt.title('Precisión de Entrenamiento y Validación del Modelo CNN')
plt.xlabel('Época')
plt.ylabel('Precisión')
plt.grid(True)

plt.tight_layout()
plt.show()
```

Figura 17. Código de la gráfica de precisión del CNN

Se evaluó el modelo CNN generando predicciones con cnn_model.predict() y aplicando un umbral de 0.5 para obtener clases binarias. Con estas predicciones se construyó y visualizó la matriz de confusión mediante confusion_matrix y ConfusionMatrixDisplay, permitiendo analizar cuantitativamente el rendimiento del modelo a través de los valores de VP, VN, FP y FN.

```
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Obtener las predicciones del modelo CNN en el conjunto de prueba
cnn_predictions = cnn_model.predict(X_test_cnn)

# Convertir las predicciones a clases binarias (0 o 1)
# Un umbral común para una activación sigmoide es 0.5
cnn_pred_classes = (cnn_predictions > 0.5).astype(int)

# Calcular la matriz de confusión
cm_cnn = confusion_matrix(y_test, cnn_pred_classes)

# Mostrar la matriz de confusión
disp = ConfusionMatrixDisplay(confusion_matrix=cm_cnn, display_labels=['Mujer', 'Hombre'])
disp.plot(cmap=plt.cm.Blues)
plt.title('Matriz de Confusión del Modelo CNN')
plt.xlabel('Etiqueta Predicha')
plt.ylabel('Etiqueta Verdadera')
plt.show()

print("Matriz de Confusión del Modelo CNN:")
print(cm_cnn)
print("\nDonde:")
print("  - True Negative (TN):", cm_cnn[0, 0], "(Mujeres clasificadas correctamente como Mujer)")
print("  - False Positive (FP):", cm_cnn[0, 1], "(Hombres clasificados incorrectamente como Mujer)")
print("  - False Negative (FN):", cm_cnn[1, 0], "(Mujeres clasificadas incorrectamente como Hombre)")
print("  - True Positive (TP):", cm_cnn[1, 1], "(Hombres clasificadas correctamente como Hombre)")
```

Figura 18. Código de la matriz de confusión del CNN

- Descripción del entrenamiento CNN

El modelo CNN fue entrenado con espectrogramas monitoreando las curvas de accuracy en cada época. Su desempeño se evaluó sobre el conjunto de prueba mediante la matriz de confusión para identificar aciertos y errores de clasificación, y se realizó una prueba final con un audio nuevo para verificar su capacidad de predicción en un caso real.

- Valoraciones del entrenamiento CNN

El modelo CNN mostró una mejora progresiva desde una precisión inicial de 0.45 hasta valores cercanos a 0.98 en entrenamiento, mientras la validación se mantuvo estable entre 0.95 y 1.00, reflejando una convergencia adecuada y buena capacidad de generalización sin evidencia de sobreajuste.

```
Epoch 1/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 7s 348ms/step - accuracy: 0.4571 - loss: 25.4304 - val_accuracy: 0.5000 - val_loss: 1.6457
Epoch 2/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.5857 - loss: 0.9848 - val_accuracy: 0.5000 - val_loss: 0.6600
Epoch 3/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.7857 - loss: 0.4223 - val_accuracy: 0.9500 - val_loss: 0.2072
Epoch 4/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.9357 - loss: 0.1479 - val_accuracy: 0.9500 - val_loss: 0.1466
Epoch 5/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.8643 - loss: 0.3857 - val_accuracy: 0.8500 - val_loss: 0.3320
Epoch 6/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9643 - loss: 0.1130 - val_accuracy: 0.9250 - val_loss: 0.1425
Epoch 7/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.9857 - loss: 0.0520 - val_accuracy: 0.9000 - val_loss: 0.2175
Epoch 8/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9714 - loss: 0.0551 - val_accuracy: 0.8000 - val_loss: 0.4704
Epoch 9/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.9857 - loss: 0.0564 - val_accuracy: 0.9500 - val_loss: 0.2247
Epoch 10/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0149 - val_accuracy: 0.9500 - val_loss: 0.0861
```

Figura 19. Valoraciones del entrenamiento CNN

- Grafica de la precisión del entrenamiento y validación del modelo CNN

La gráfica de precisión del modelo CNN evidencia un aprendizaje progresivo y eficiente durante las 10 épocas, alcanzando valores cercanos a 1.00 en entrenamiento y alrededor de 0.95 en validación. La cercanía entre ambas curvas indica una buena capacidad de generalización del modelo sobre datos no vistos. Aunque existe una ligera fluctuación en la validación en épocas intermedias.

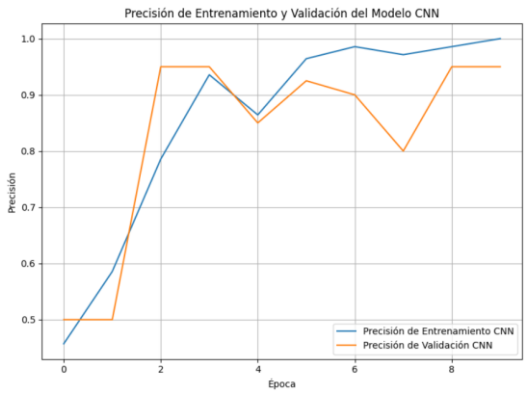

Figura 20. Grafica de la precisión del CNN

- Matriz de confusión del modelo CNN

La matriz de confusión del modelo CNN muestra un desempeño altamente preciso en la clasificación de voces, logrando identificar correctamente 9 de 10 audios femeninos y la totalidad de los audios masculinos evaluados. Solo se presenta un error de clasificación, donde una voz femenina fue predicha como masculina, mientras que no se registraron falsos negativos para la clase masculina. Evidencia la capacidad para distinguir patrones acústicos entre ambas clases.

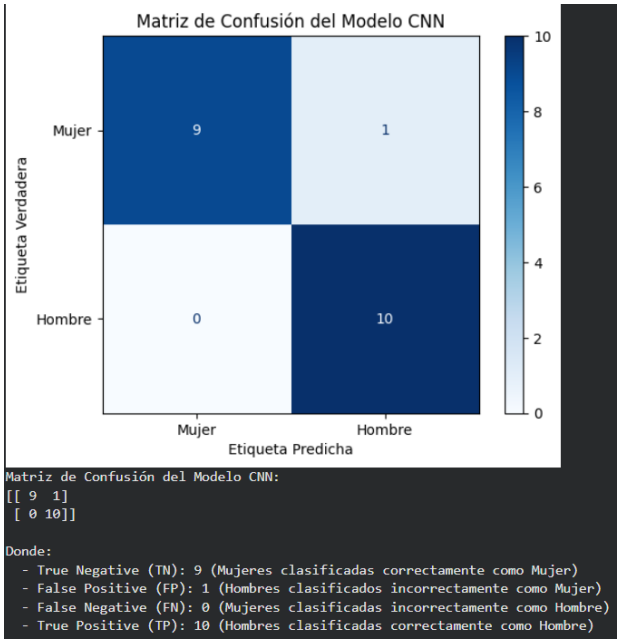

Figura 21. Matriz confusión del CNN

H. Prueba final del modelo CNN con audio nuevo

- Código implementado

Se evalúa el modelo CNN utilizando la función evaluate() sobre el conjunto de prueba X_test_cnn y sus etiquetas y_test, obteniendo dos métricas: cnn_loss, que representa el nivel de error del modelo, y cnn_acc, que indica la precisión alcanzada en la clasificación. Posteriormente, se selecciona un audio del conjunto de prueba mediante X_test[1] para realizar una predicción individual con un archivo no visto durante el entrenamiento.

Prueba final del modelo CNN con un audio nuevo

```
cnn_loss, cnn_acc = cnn_model.evaluate(X_test_cnn, y_test)
print("Precisión final CNN:", cnn_acc)

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 309ms/step - accuracy: 0.9500 - loss: 0.1422
Precisión final CNN: 0.949999988079071
```

cnn_loss qué tanto se equivoca ; **cnn_acc** qué tan preciso es

```
# ================ PRUEBA FINAL CNN ================

# Elegir un audio de prueba (del conjunto que el modelo NO vio)
audio_prueba = X_test[1]

print("Audio usado:", audio_prueba)

# Convertir audio igual que en entrenamiento
spec = audio_to_melspectrogram(audio_prueba)
spec = spec[np.newaxis, ..., np.newaxis]

# Predicción
pred = cnn_model.predict(spec)

print("Valor predicción:", pred)

# Resultado interpretado
if pred > 0.5:
    print("Predicción CNN: Hombre")
else:
    print("Predicción CNN: Mujer")
```

Figura 22. Prueba final del modelo

La predicción se realiza con cnn_model.predict(spec), generando una probabilidad entre 0 y 1. A partir de un umbral de 0.5, se interpreta el resultado como voz masculina o femenina mediante una estructura condicional if.

```
# Mostrar etiqueta real
if "female" in audio_prueba:
    print("Etiqueta real: Mujer")
else:
    print("Etiqueta real: Hombre")
```

Figura 23. Denominación de las etiquetas

Se compara la predicción con la etiqueta real del audio verificando si la ruta contiene la palabra "female", y se utiliza Audio(audio_prueba) de IPython.display para reproducir el archivo de voz directamente en el entorno de ejecución.

```
from IPython.display import Audio
Audio(audio_prueba)
```

0:00 / 0:03

Figura 24. Importar audio

- Análisis de los resultados

La prueba final muestra que el modelo CNN clasificó correctamente el audio masculino con una probabilidad de 0.99, evidenciando una alta confianza.

```
Audio usado: data/male/arctic_a0378(3).wav
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 454ms/step
Valor predicción: [[0.99119455]]
Predicción CNN: Hombre
Etiqueta real: Hombre
```

Figura 25. Resultados CNN

I. Preparación de datos para el modelo RNN

- *Código implementado*

Se definió la función audio_to_mfcc para convertir cada archivo de audio en una matriz de coeficientes MFCC, cargando la señal con librosa.load a 22 050 Hz y extrayendo 13 coeficientes mediante librosa.feature.mfcc. Para garantizar dimensiones uniformes, se aplicó padding con np.pad cuando la señal era más corta que 130 tramas, o recorte automático cuando la excedía, obteniendo matrices de forma (13×130) por audio.

Modelo RNN 🌸

🔹 Preparación de datos para el modelo RNN - RNN procesa la señal como una secuencia temporal, manteniendo memoria de los valores anteriores para entender la evolución del audio."

En esta sección los audios se convierten en características MFCC, las cuales representan el sonido como una secuencia de valores. Esta representación es adecuada para modelos RNN porque permite analizar cómo cambia la voz a través del tiempo.

```
def audio_to_mfcc(path, max_pad_len=130):
    audio, sr = librosa.load(path, sr=22050)
    mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=13)

    if mfcc.shape[1] < max_pad_len:
        pad_width = max_pad_len - mfcc.shape[1]
        mfcc = np.pad(mfcc, pad_width=((0,0), (0,pad_width)), mode='constant')
    else:
        mfcc = mfcc[:, :max_pad_len]

    return mfcc
```

Figura 26. Código de preparación

Finalmente, la función fue aplicada sobre los conjuntos de entrenamiento, validación y prueba, almacenando los resultados en arreglos NumPy listos para ser procesados por el modelo RNN.

Conversión de audios a MFCC son una representación numérica del audio que captura las características más importantes de la voz humana.

```
X_train_rnn = np.array([audio_to_mfcc(p) for p in X_train])
X_val_rnn = np.array([audio_to_mfcc(p) for p in X_val])
X_test_rnn = np.array([audio_to_mfcc(p) for p in X_test])

print("Train RNN:", X_train_rnn.shape)
print("Val RNN:", X_val_rnn.shape)
print("Test RNN:", X_test_rnn.shape)

Train RNN: (140, 13, 130)
Val RNN: (40, 13, 130)
Test RNN: (20, 13, 130)
```

Figura 27. Código de conversión de audios

J. Construcción del modelo RNN

- Código implementado

Se construyó una RNN con TensorFlow Keras mediante una arquitectura secuencial compuesta por una capa LSTM de 64 unidades que recibe secuencias de forma (13×130), seguida de una capa densa de 32 neuronas con activación ReLU y una capa de salida con activación sigmoide para clasificación binaria. El modelo se compiló con el optimizador Adam, función de pérdida binary_crossentropy y métrica accuracy.

🔷 Construcción y configuración del modelo RNN

```
from tensorflow.keras import models, layers

rnn_model = models.Sequential([
    layers.LSTM(64, input_shape=(13,130)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

rnn_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

rnn_model.summary()
```

Figura 28. Código de construcción de RNN

- Descripción de la construcción

El modelo RNN implementado utiliza una arquitectura LSTM de 64 unidades que procesa secuencias MFCC de forma (13×130), capturando dependencias temporales de la voz. La salida de la LSTM se conecta a una capa densa de 32 neuronas con activación ReLU para el aprendizaje de características, y una capa de salida con activación sigmoide que produce la probabilidad de clasificación binaria. El modelo se configuró con el optimizador Adam, función de pérdida binary_crossentropy y se entrenó durante 10 épocas con un batch size de 16.

- Tabla de tipos de parámetros en RNN

La arquitectura RNN concentra la mayor parte de sus parámetros en la capa LSTM de 64 unidades con 49 920 parámetros, encargada de capturar las dependencias temporales de la secuencia MFCC. Su salida alimenta una capa densa de 32 neuronas con 2 080 parámetros y una capa de salida con 33 parámetros para la clasificación binaria. El modelo totaliza 52 033 parámetros entrenables.

Model: "sequential_2"

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 64)	49,920
dense_4 (Dense)	(None, 32)	2,080
dense_5 (Dense)	(None, 1)	33

Total params: 52,033 (203.25 KB)
Trainable params: 52,033 (203.25 KB)
Non-trainable params: 0 (0.00 B)

Figura 29. Tabla de parámetros RNN

K. Entrenamiento del modelo RNN

- Código implementado

El modelo RNN se entrenó con el método fit() durante 10 épocas y un batch size de 16, usando las secuencias

MFCC como entrada. En cada época se evaluó el desempeño sobre el conjunto de validación sin modificar los pesos, permitiendo monitorear la capacidad de generalización del modelo.

🔷 Entrenamiento del modelo RNN

```
history_rnn = rnn_model.fit(
    X_train_rnn, y_train,
    validation_data=(X_val_rnn, y_val),
    epochs=10,
    batch_size=16
)
```

Figura 30. Código de entrenamiento del modelo RNN

Mediante matplotlib se graficaron las curvas de precisión de entrenamiento y validación extraídas del objeto history_rnn, visualizando la evolución del aprendizaje a lo largo de las épocas.

Se generaron predicciones con rnn_model.predict() aplicando un umbral de 0.5 para obtener clases binarias, con las cuales se construyó y visualizó la matriz de confusión mediante confusion_matrix y ConfusionMatrixDisplay, permitiendo analizar cuantitativamente el rendimiento del modelo a través de los valores de VP, VN, FP y FN.

```
import matplotlib.pyplot as plt

# Obtener los datos de historial para el modelo RNN
acc_rnn = history_rnn.history['accuracy']
val_acc_rnn = history_rnn.history['val_accuracy']
epochs_range = range(len(acc_rnn))

plt.figure(figsize=(8, 6))

# Gráfico para el modelo RNN
plt.plot(epochs_range, acc_rnn, label='Precisión de Entrenamiento RNN')
plt.plot(epochs_range, val_acc_rnn, label='Precisión de Validación RNN')
plt.legend(loc='lower right')
plt.title('Precisión de Entrenamiento y Validación del Modelo RNN')
plt.xlabel('Época')
plt.ylabel('Precisión')
plt.grid(True)

plt.tight_layout()
plt.show()
```

Figura 31. Código de la gráfica de precisión del RNN

```
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Obtener las predicciones del modelo RNN en el conjunto de prueba
rnn_predictions = rnn_model.predict(X_test_rnn)

# Convertir las predicciones a clases binarias (0 o 1)
# Un umbral común para una activación sigmoide es 0.5
rnn_pred_classes = (rnn_predictions > 0.5).astype(int)

# Calcular la matriz de confusión
cm_rnn = confusion_matrix(y_test, rnn_pred_classes)

# Mostrar la matriz de confusión
disp = ConfusionMatrixDisplay(confusion_matrix=cm_rnn, display_labels=['Mujer', 'Hombre'])
disp.plot(cmap=plt.cm.Blues)
plt.title('Matriz de Confusión del Modelo RNN')
plt.xlabel('Etiqueta Predicha')
plt.ylabel('Etiqueta Verdadera')
plt.show()

print("Matriz de Confusión del Modelo RNN:")
print(cm_rnn)
print("\nDonde:")
print("  - True Negative (TN):", cm_rnn[0, 0], "(Mujeres clasificadas correctamente como Mujer)")
print("  - False Positive (FP):", cm_rnn[0, 1], "(Hombres clasificados incorrectamente como Mujer)")
print("  - False Negative (FN):", cm_rnn[1, 0], "(Mujeres clasificadas incorrectamente como Hombre)")
print("  - True Positive (TP):", cm_rnn[1, 1], "(Hombres clasificadas correctamente como Hombre)")
```

Figura 32. Código de la gráfica de confusión del RNN

- Descripción del entrenamiento RNN

El modelo RNN fue entrenado durante 10 épocas, mostrando curvas de aprendizaje con una precisión de entrenamiento que alcanzó 1.00, mientras la validación se estabilizó alrededor de 0.625, evidenciando sobreajuste. La evaluación sobre el conjunto de prueba mediante la matriz de confusión reveló dificultades para generalizar en ambas clases, y la predicción final sobre un audio nuevo clasificó incorrectamente una voz femenina como masculina con una probabilidad de 0.89, confirmando las limitaciones de generalización del modelo.

- Valoraciones del entrenamiento RNN

El modelo CNN mostró una mejora acelerada desde una precisión inicial de 0.58 hasta alcanzar 1.00 en entrenamiento desde la época 6, mientras que la validación se mantuvo entre 0.50 y 0.625, evidenciando una brecha considerable entre ambas curvas que indica sobreajuste y una capacidad de generalización limitada.

```
Epoch 1/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 3s 61ms/step - accuracy: 0.5786 - loss: 0.6812 - val_accuracy: 0.5000 - val_loss: 0.7121
Epoch 2/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9143 - loss: 0.4444 - val_accuracy: 0.4750 - val_loss: 0.7026
Epoch 3/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9786 - loss: 0.3396 - val_accuracy: 0.5500 - val_loss: 0.6785
Epoch 4/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9786 - loss: 0.2583 - val_accuracy: 0.5500 - val_loss: 0.6664
Epoch 5/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9929 - loss: 0.1962 - val_accuracy: 0.6000 - val_loss: 0.6565
Epoch 6/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 1.0000 - loss: 0.1499 - val_accuracy: 0.6000 - val_loss: 0.6531
Epoch 7/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 1.0000 - loss: 0.1168 - val_accuracy: 0.6000 - val_loss: 0.6495
Epoch 8/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 1.0000 - loss: 0.0903 - val_accuracy: 0.6250 - val_loss: 0.6411
Epoch 9/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 1.0000 - loss: 0.0714 - val_accuracy: 0.6250 - val_loss: 0.6442
Epoch 10/10
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 1.0000 - loss: 0.0559 - val_accuracy: 0.6250 - val_loss: 0.6534
```

Figura 33. Parámetros del RNN

- Grafica de la precisión del entrenamiento y validación del modelo RNN

La gráfica de precisión del modelo RNN evidencia un aprendizaje acelerado en las primeras épocas, alcanzando valores cercanos a 1.00 en entrenamiento desde la época 2 y manteniéndose estable hasta la época 9. Sin embargo, la curva de validación parte de 0.50, desciende brevemente hasta 0.47 en la época 1 y se recupera progresivamente hasta estabilizarse alrededor de 0.625 desde la época 6, manteniéndose considerablemente por debajo del entrenamiento durante todo el proceso.

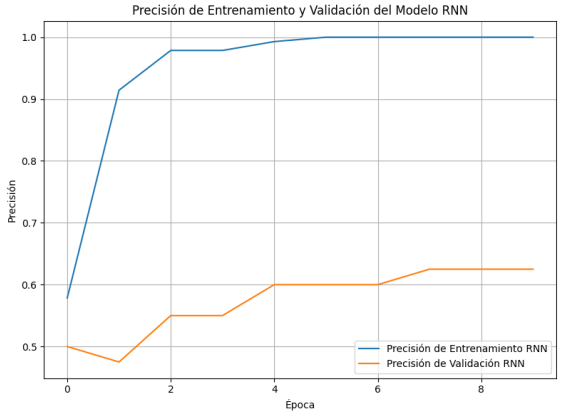

Figura 34. Grafica de precisión de RNN

- Matriz de confusión del modelo RNN

La matriz de confusión del modelo RNN muestra un desempeño asimétrico entre clases: identificó correctamente 8 de 10 voces masculinas y 7 de 10 voces femeninas, generando 3 falsos positivos donde audios femeninos fueron clasificados como masculinos y 2 falsos negativos donde voces masculinas fueron predichas como femeninas. Esto refleja dificultades del modelo para distinguir patrones acústicos en ambas clases, consistente con el sobreajuste observado en las curvas de entrenamiento.

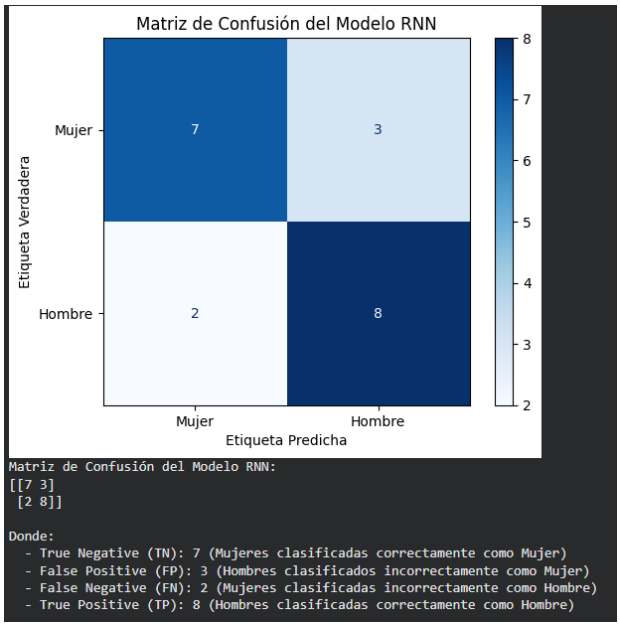

Figura 35. Matriz de confusión de RNN

L. Prueba final del modelo RNN con audio nuevo

- Código implementado

Se evalúa el modelo RNN con evaluate() obteniendo las métricas rnn_loss y rnn_acc sobre el conjunto de prueba. Para la predicción final se selecciona X_test[10], se extraen sus coeficientes MFCC con audio_to_mfcc() y se añade una dimensión con np.newaxis para compatibilidad con el modelo.

La predicción se genera con rnn_model.predict(), interpretando el resultado con un umbral de 0.5 para determinar si la voz corresponde a hombre o mujer. Finalmente, se compara con la etiqueta real verificando la ruta del archivo y se reproduce el audio con IPython.display para validar auditivamente la predicción.

```
rnn_loss, rnn_acc = rnn_model.evaluate(X_test_rnn, y_test)
print("Precisión final RNN:", rnn_acc)

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - accuracy: 0.8000 - loss: 0.4814
Precisión final RNN: 0.800000011920929

# PRUEBA FINAL RNN

audio_prueba = X_test[10]

print("Audio usado:", audio_prueba)

mfcc = audio_to_mfcc(audio_prueba)
mfcc = mfcc[np.newaxis, ...]

pred = rnn_model.predict(mfcc)

print("Valor predicción:", pred)

if pred > 0.5:
    print("Predicción RNN: Hombre")
else:
    print("Predicción RNN: Mujer")

if "female" in audio_prueba:
    print("Etiqueta real: Mujer")
else:
    print("Etiqueta real: Hombre")
```

Figura 36. Código prueba final

```
from IPython.display import Audio
Audio(audio_prueba)
```

0:00 / 0:03

Figura 37. Código para reproducir

- Análisis de los resultados

La prueba final muestra que el modelo RNN clasificó incorrectamente el audio, prediciendo "Hombre" con un valor de 0.522. Confirmando las limitaciones de generalización del modelo.

```
Audio usado: data/female/arctic_a0062(7).wav
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 224ms/step
Valor predicción: [[0.5223215]]
Predicción RNN: Hombre
Etiqueta real: Mujer
```

Figura 38. Resultados RNN

J. Comparación entre el modelo CNN y RNN

La comparación entre ambos modelos evidencia una superioridad del CNN, que clasificó correctamente una voz masculina con una probabilidad de 0.99, frente al RNN que clasificó incorrectamente una voz femenina como masculina con una probabilidad de 0.52. Las matrices de confusión confirman que el CNN clasifica con mayor equilibrio ambas clases, mientras el RNN muestra mayor dificultad con voces femeninas. En tiempo de entrenamiento el CNN resultó más eficiente, sugiriendo que la representación mediante mel-

espectrogramas es más robusta y adecuada para la clasificación de género por voz que el enfoque secuencial con MFCC.

III. CONCLUSION

El desarrollo del proyecto permitió comparar dos enfoques de clasificación automática de género por voz, evidenciando una superioridad clara del CNN sobre el RNN. El CNN alcanzó una precisión de validación entre 0.95 y 1.00 frente al 0.625 del modelo recurrente, demostrando que la representación visual mediante mel-espectrogramas captura más efectivamente los patrones acústicos relevantes. El modelo RNN presentó sobreajuste y mayor dificultad para distinguir voces femeninas, generando confusiones atribuibles a la similitud en frecuencias fundamentales entre clases, presencia de ruido en las grabaciones, variaciones en duración y energía espectral, y al tamaño reducido del dataset, factores que limitaron su capacidad de generalización.

Desde una perspectiva técnica, el CNN resulta más adecuado cuando el audio puede representarse como imagen tiempo-frecuencia, ofreciendo mayor eficiencia y generalización en clasificación binaria con patrones espectrales definidos. El RNN es más conveniente cuando se requiere modelar dependencias temporales estrictas, como reconocimiento de palabras o frases completas. Este proyecto demostró que la integración entre el procesamiento digital de señales y el aprendizaje profundo es fundamental, ya que la calidad de la representación de entrada determina directamente el rendimiento del modelo, siendo la etapa de extracción de características tan importante como la arquitectura de la red neuronal utilizada.

REFERENCIAS

[1] A. V. Oppenheim y R. W. Schafer, Tratamiento de Señales en Tiempo Discreto, 3.ª ed. Upper Saddle River, NJ: Prentice Hall, 2010.

[2] J. G. Proakis y D. G. Manolakis, Tratamiento Digital de Señales, 4.ª ed. Madrid: Prentice Hall, 2007.

[3] I. Goodfellow, Y. Bengio y A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.

[4] S. Hochreiter y J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[5] Y. LeCun, L. Bottou, Y. Bengio y P. Haffner, "Gradient-based learning applied to document recognition," Proc. IEEE, vol. 86, no. 11, pp. 2278–2324, nov. 1998.

[6] B. McFee et al., "librosa: Audio and music signal analysis in Python," en Proc. 14th Python in Science Conf. (SciPy), Austin, TX, 2015, pp. 18–25.